



S.G. Ganesh

A Bug or a Feature?

A puzzling aspect of bugs is that they often turn out to be features (and vice versa)! Let's explore this interesting topic with an example.

In my experience working with enterprise software, I come across numerous reports by users about bugs in the software. On detailed analysis, such 'bug reports' often turn out to actually be 'features' of the software, or misunderstood functionality of the software. It is difficult to give a generic example of this problem, since it is very specific to the context of the application. However, I recently came across an example that anyone can relate to.

I was trying to rotate an image in MS PowerPoint. We can right-click on an image and type the rotation degree in the text-box that pops up (see Figure 1). By mistake, when I typed a degree outside the range -360° to $+360^\circ$, a help message popped up. I thought it was asking me to "Enter a value from -360° to 360° ", but on looking closer, I realised it read, "Enter a value from -3600° to 3600° !" I thought it was a bug in both the help message and the software, because the text-box accepted the range of values from -3600° to $+3600^\circ$. However, when I tried typing outside the range -3600° or $+3600^\circ$, the software rounded the degree to -3600° or $+3600^\circ$ as appropriate, and used the modulus of 360 to determine the rotation angle. So, it was clear that this was not a 'bug', but an intended 'feature' in the software!

So, in general, should this be considered a bug or a feature? We all know that you can rotate an image from -360° to $+360^\circ$; for angle values outside this range, the rotation angle is the same as the angle value mod 360. So, when we try rotating the image in software, we expect two possibilities: either the software limits the range from -360° to $+360^\circ$, or it accepts any range of values, and applies mod 360 to determine the rotation angle. However, it is unintuitive and unexpected that the allowed range is from -3600° or $+3600^\circ$, in which the acceptable angles are 10 times the range. Specifically, the range of acceptable angle values is 10 times (an arbitrary limitation assumed by the software) the expected range, which is incorrect. Hence, it is a bug in the software. However, if you post this bug to the developers of the software, they will reject this as a bug request. Why? From the software vendor's perspective, this range value was thought about, and extra range (10 times the expected range) is allowed as a 'feature', and hence it's not a bug!

In my work experience, I remember many heated debates with customers (or even within development teams) about whether the given problem was a bug or a feature. The problem becomes complicated, and results in a strange situation where

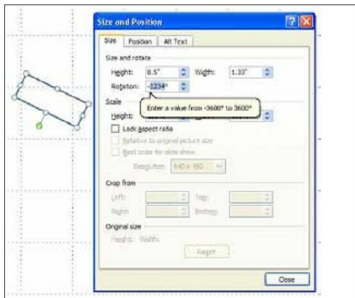


Figure 1: A 'bug vs feature' example from MS PowerPoint

the bug is used as a feature! Assume that the software you have shipped has a bug. The customers don't know that it is a bug, but think it is a feature, and start using it. Now, even if you realise that it is a bug, you cannot fix it. Why? Since customers think it is a feature, if that feature is not available anymore in the new version, they will start complaining in a big way. Hence, you would need to support that 'bug' in future releases too. This requirement is known as 'bug compatibility': you not only need to maintain compatibility with the old 'features' supported by the software, but also maintain compatibility with old 'bugs' in the newer version of the software!

This 'bug compatibility' problem is very pronounced in APIs (Application Programming Interfaces). For example, once a library or a framework is publicly available, you cannot fix the bugs that were introduced in the earlier releases. Doing so would break the compatibility with the existing users who are still using the old version of the library. This is one of the reasons why API development is a challenging task; we need to get it right the first time, or else we'll never be able to fix it to get it right. 

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bangalore. You can reach him at sgganesh@gmail.com.